Протокол проверок и нагрузочный отчёт

Актуальная повторная проверка 20.10.2026

Финальные результаты remediation имеют приоритет над историей ниже: REGRESSION-RESULTS, QUALITY-REPORT, RELEASE-READINESS. Выполнено 38 unit/security/API/eval тестов, coverage 84.54%; critical lint и шесть mypy modules настроены и запускаются. CPU/GPU integration, tokenizer budget, index rollback/restore и offline wheels имеют отдельные машинные evidence.

Учебный synthetic MVP: **GO**. Промышленный запуск HOLD. Старые числа и команды ниже - история 05-08.10; сведения «lint не настроен», «индексы пересоздаются» и старый status контейнеров к текущей реализации не относятся. Путь актуального benchmark: Docker client → gateway:18080 → backend, отдельные CPU300/8 и GPU48/4. Сырые результаты и ограничения в remediation, не ASGI throughput. Восстановление тестового snapshot: 0.6144 s, production RPO/RTO не установлены.

Исходный протокол: 05.10.2026; независимое обновление: 08.10.2026. Реальные результаты локального стенда; данные предприятия не использовались.

Независимая контрольная проверка 08.10.2026

Новый аудит выполнен по исходным заданиям и материалам курса. Исторические измерения ниже сохранены отдельно. Полная текущая матрица - ../../Матрица-2026-10-08.md, отчёт - ../../Аудит-2026-10-08.md.

Исправлены проверка известной injection после NFKC-нормализации (query и ingestion), обработка отказов Qdrant через HTTP 503 и фиксированные даты в integration/model-manifest scripts. Добавлены регрессионные assertions. Независимый набор до правок: 3/6 PASS; после: 6/6 PASS. Штатный backend.check, pip check, compileall и backend.integration_check в CPU/GPU профилях прошли.

Профиль / путь	Запросы / concurrency	Успешный RPS	p50 / p95 / p99, мс	Ошибки
Docker HTTP client → gateway:18080 → backend, extractive + Qdrant/SQL	300 / 8	8,95	853,58 / 1121,98 / 1324,50	0
Тот же путь → vLLM Qwen2.5-7B AWQ Marlin, RTX 4070	48 / 4	2,46	1603,87 / 1996,73 / 2006,53	0

Это замер пропускной способности короткого закрытого сценария с повторяющимся вопросом, не поиск предельной мощности. GPU запросы выполнялись после интеграционного прогрева, prefix cache включён. Прогон CPU завершён до запуска GPU-нагрузки. Клиент внутри backend-контейнера; запрос проходит реальный Nginx gateway по Docker-сети, не Windows loopback. Поле scope штатного benchmark содержит общий шаблон про loopback; фактическая команда ниже уточняет путь. Аппаратно подтверждены RTX 4070 / 12282 MiB / driver 616.92. Малый синтетический корпус, локальный bind mount и workload ограничивают переносимость цифр.

Повторный контроль секретного бюджета проекта П-9: A получил точную цитату и сумму 987654321, B - no_evidence без вызова LLM. Это синтетическая фикстура, не реальные финансовые данные. Все 11 файлов локальных весов совпали с SHA256-манифестом. Реальные OCR, Qdrant ACL, Neo4j, Jaeger, Prometheus и блокировка исходящего TCP backend повторно проверены интеграционным тестом.

Данные этого аудита: evidence/audit-2026-10-08/benchmark-cpu.json, benchmark-gpu.json, integration-cpu.json, integration-gpu.json, gpu-secret-check.json, model-hashes.json, edge-before.json, edge-after.json.

Команды из корня monorepo; для GPU добавить -f infra/compose.gpu.yaml ко всем compose-вызовам:

docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml build backend
docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml up -d --no-build
docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml exec -T backend python -m backend.check
docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml exec -T backend pip check
docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml exec -T -e PYTHONPYCACHEPREFIX=/tmp/audit-pycache backend python -m compileall -q backend
docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml exec -T backend python -m backend.integration_check
docker compose -f infra/compose.yaml -f infra/compose.indexes.yaml exec -T backend python -m backend.benchmark --url http://gateway:18080 --requests 300 --concurrency 8 --output /app/data/audit-benchmark-cpu.json

GPU benchmark: тот же вызов с GPU compose, --requests 48 --concurrency 4 --output /app/data/audit-benchmark-gpu.json. Отдельные lint/typecheck команды в проекте не настроены; они не отмечаются как PASS. Production corpus, отказоустойчивость, security certification, полноценный red teaming и LLM-as-Judge Golden Set не проверены.

При подготовке аудита был ошибочный вызов --url http://gateway без порта: 300/300 сетевых ошибок, это не корректный нагрузочный прогон. После сверки nginx.conf использован порт 18080; неудачный JSON сохранён в корневом каталоге аудита. Native SQLite-тест в sandbox не запускался из-за доступа к temp; тот же код успешно проверен в штатном Docker-окружении.

Независимая контрольная проверка 05.10.2026

Ниже исходный протокол предыдущего исполнителя сохранён как история. Его заявления не использовались как доказательства: курс и задания перечитаны, код/контракты/формулы проверены заново. Матрица всех требований находится в ../../Независимая-проверка-матрица.md, итоговый отчёт - в ../../Независимая-проверка.md.

Подтверждены и исправлены: некорректные ответы модели (JSON/структура/token usage), не-ASCII Bearer, нехэшируемый chunk ID, ожидание первого Prometheus scrape в integration check. Сначала сохранены падающие пробы, затем внесены минимальные guards и повторены проверки. backend.check дополнен регрессиями. Deployment теперь явно показывает проектную балансировку готовых CPU/GPU-реплик; несколько API-реплик требуют общей SQL/history, отдельные SQLite недопустимы. Горячая синхронизация индексов не заявляется: после ingestion/ACL updates нужен restart backend, порядок указан в README.

Новый независимый прогон	Запросов / concurrency	Успешных RPS	p50 / p95 / p99, мс	Ошибок
Windows native ASGI, fresh isolated corpus, extractive	300 / 8	59,11	124,46 / 272,04 / 319,80	0
Docker ASGI, рабочая SQL в Windows bind mount, extractive	300 / 8	9,35	843,72 / 976,06 / 1072,61	0
Windows HTTP loopback gateway → backend → vLLM AWQ Marlin	24 / 4	2,20	1706,62 / 2342,36 / 2353,48	0

Новые JSON - evidence/independent-benchmark-cpu-native.json, independent-benchmark-cpu-docker.json, independent-benchmark-gpu.json. CPU окружения и состояния корпусов различаются, поэтому сравнение их RPS не оценивает ускорение модели. GPU workload совпадает с контрольным вопросом проекта; p95 <5 с воспроизведён. 6,31× для historical generic AWQ/Marlin не объявляется новым независимым экспериментом: generic kernel повторно не запускался.

Свежие backend.check, backend.integration_check в CPU и GPU профилях, pip check, compileall с cache в /tmp, Compose config/build/up прошли. Проверены все 118 Excel formulas, OpenAPI 3.1 и 5 schema examples, реальные LangGraph переходы notebook, 69 original hashes, pinned official Qwen artifact hashes. Видео полностью декодировано ffmpeg, duration 374,046 с; выборочные главы OCR/RBAC/Jaeger/vLLM просмотрены. Проверки не доказывают production capacity, сложную

геометрию чертежей или полноценный red teaming. GPU после проверки остановлен, финальный backend оставлен в extractive-профиле с индексами/наблюдаемостью.

Стенд и профиль

Windows 11, Docker Desktop / Linux containers, Intel Core i7-14700, 68 402 135 040 bytes физической RAM (63,7 GiB), NVIDIA RTX 4070 12 282 MiB, driver 616.92. GPU разделяется с приложениями рабочего стола. Backend Python 3.12 / FastAPI / LangGraph 1.2.12; полный tested dependency lock приложен. Реальные Qdrant 1.19.1, Neo4j 5.26, Jaeger 1.76, Prometheus 3.5, Grafana 12.1 и Nginx gateway.

vLLM 0.10.2, Qwen2.5-7B-Instruct-AWQ, готовое INT4 artifact с проверенными LFS SHA-256, FP16 KV, context cap 4096, max-num-seqs 4, max-num-batched-tokens 1024, gpu-memory-utilization 0,80, prefix caching. Финальный kernel AWQ Marlin: weights 5,2039 GiB, доступно 3,35 GiB KV / 62 640 tokens. Логовая concurrency 15,29 - расчёт памяти, scheduler фактически ограничен четырьмя запросами. Все inference выполнялись внутри сети Compose; исходящий TCP из backend заблокирован фактическим тестом.

Фактическая нагрузка

Путь	Запросов / concurrency	Успешных RPS	p50 / p95 / p99, мс	Ошибок
CPU extractive, in-process ASGI	300 / 8	58,27	129,24 / 163,73 / 347,84	0
HTTP gateway → backend → GPU, generic AWQ	24 / 4	0,36	10 995,80 / 11 659,09 / 11 669,01	0
HTTP gateway → backend → GPU, AWQ Marlin	24 / 4	2,27	1675,28 / 1964,63 / 1972,34	0

Исходные JSON: benchmark.json, benchmark-awq.json, benchmark-gpu.json. GPU elapsed 66,721 с → 10,567 с. Прогоны измеряют полную задержку разрешённого ответа с проверкой exact quotes; streaming TTFT не измерялся, потому что SSE выпускает только завершённый проверенный результат. Установленный vLLM прямо рекомендовал совместимый Marlin вместо generic AWQ; baseline сохранён. Показанное ускорение около 6,31 раза по elapsed относится к этому стенду и workload.

Вход - одинаковый вопрос User B о связи Н-17 → К-2 → Д-8, три синтетических документа, тёплая модель и prefix cache. Нет случайного корпуса, широкого распределения длин, soak test, отказов оборудования и конкурентного ingestion. Поэтому это достигнутая скорость короткого контрольного прогона, а не максимальная capacity / production SLA. Цель p95 ≤5 с на этом сценарии выполнена только финальным kernel. CPU число не измеряет нейросетевую генерацию и не сравнивается с GPU как равнозначная нагрузка. Для sizing реального пилота нужны репрезентативный corpus, query mix, Golden Set и длинный ступенчатый load test.

Выполненные проверки

- python -m backend.check: PASS - 2-hop provenance, tenant/RBAC/shared entity, cross-user history, отзыв grant во время generation, PII masking, poisoned documents, запрет внешнего inference URL, invented quote, version update, HTTP и проверенный SSE.

- python -m backend.integration_check внутри Docker: PASS - реальная Qwen generation, User B без бюджета, попытка передать роли получает 422, отсутствие auth 401, чужая session 403, injection blocked, outsider no_evidence, OCR русского image-only PDF, Qdrant ACL filter, Neo4j mirror, Jaeger spans, Prometheus metrics и отсутствие backend egress. Итог в evidence/integration-result.json.

- docker compose ... build backend: PASS с requirements-lock.txt; базовый образ и локальные weights получены на staging. Runtime не скачивает модель. docker compose ... config и фактический запуск полного профиля выполнены.

- Браузер: User A получает синтетический секретный бюджет; тот же вопрос User B даёт no_evidence. User B получает ровно две разрешённые цитаты для 2-hop вопроса. Реальный Jaeger trace содержит 13 spans, включая llm.generate. В Neo4j 10 nodes / 12 relations, из них 3 CO_MENTION.

- ДЗ 3: исполнен LangGraph Manager → Searcher → Manager с проверкой budget/missing/empty; outputs notebook соответствуют исполнению.

- ДЗ 2: OpenAPI 3.1 и JSON request/response examples проверены валидаторами.

- ДЗ 7: формулы XLSX пересчитаны; отдельно проверены FP16/INT4/KV, GPU capacity, cloud arithmetic и savings. PDF/Draw.io отрендерены и визуально проверены; исходные 69 файлов проверены SHA-256.

Самопроверка изолирована от production-derived indexes: её временный SQL не синхронизируется в рабочие Qdrant/Neo4j. При финальной проверке это исправление подтверждено запуском check, затем integration_check в одном стенде.

Воспроизведение

Из корня monorepo после init_demo и подготовки weights:

```
docker compose -p course-knowledge -f infra/compose.yaml -f infra/compose.indexes.yaml -f infra/compose.gpu.yaml up -d --build
docker compose -p course-knowledge -f infra/compose.yaml -f infra/compose.indexes.yaml -f infra/compose.gpu.yaml exec -T backend python -m backend.check
docker compose -p course-knowledge -f infra/compose.yaml -f infra/compose.indexes.yaml -f infra/compose.gpu.yaml exec -T backend python -m backend.integration_check
python -m backend.benchmark --url http://127.0.0.1:18080 --requests 24 --concurrency 4 --output docs/benchmark-gpu.json
```

Дождаться окончания model loading и пробного успешного query. /health - liveness API, не readiness всей платформы. Для generic baseline заменить quantization на awq и повторить тот же workload. Не запускать одновременно стороннюю GPU-нагрузку.

Ограничения и артефакты

Hash embeddings / lexical reranker проверены фактически; опциональный pretrained multilingual encoder не доставлен и не оценён. Канонический traversal выполняется в SQLite, Neo4j показывает производный граф. OCR подтверждён для русского текста image-only PDF; геометрия чертежей и сложные таблицы не объявлены распознанными. Regex guardrails не заменяют red teaming; exact quote validation и ACL являются независимыми слоями. Синтетический корпус не заменяет LLM-as-a-Judge / доменную оценку качества.

Vault, SSO, mTLS, DMZ, HA, backup restore и production certification остаются архитектурными controls ADD, а не выполненными развёртываниями MVP. Учебный стек и небольшой global lock имеют явные ceilings. Секреты в data/ исключены из ZIP; перенос начинается с init_demo. Weights исключены из ZIP и уже находятся локально в models/; staging script и manifest включены. Записи экранов и журналы в evidence/, шестиминутный смонтированный deep dive в демо.mp4, текст озвучки в демо-текст.md.